Flutter Testing Report

REST API App — rest_api_app

Unit Tests • Widget Tests • Mocking • Coverage

Prepared by: Developer
Date: June 12, 2026
SDK: Dart ^3.11.5 • Flutter • Mockito ^5.6.4

Table of Contents

1. Executive Summary

This report documents the complete Flutter testing implementation for the rest_api_app project. The application is a multi-feature Flutter app consuming three external REST APIs: OpenWeatherMap (weather data), REST Countries (country information), and NewsAPI (news headlines). The testing suite covers unit tests, widget tests, and mock-based integration testing.

35+	3	8	Mockito
Total Test Cases	Test Categories	Test Files	Mocking Library

The testing suite successfully covers the three core architectural layers of the application:

- Model layer: JSON parsing, field validation, and edge case handling
- Service/Controller layer: API communication, state management, and error handling
- Widget layer: UI rendering, user interaction, and visual correctness

2. Project Overview

2.1 Application Architecture

The rest_api_app follows a clean GetX-based MVC (Model-View-Controller) architecture:

Layer	Files	Responsibility
Models	weather.dart, country.dart, news.dart	Data representation and JSON parsing
Services	api_service.dart, weather_service.dart, country_service.dart, news_service.dart	HTTP communication with APIs
Controllers	weather_controller.dart, country_controller.dart, news_controller.dart	Business logic and state management
Screens	home_screen.dart, weather_screen.dart, countries_screen.dart, news_screen.dart, ...	UI presentation layer
Widgets	country_card.dart, news_card.dart	Reusable UI components

2.2 Dependencies

The project uses the following key packages:

Package	Version	Purpose
flutter / flutter_test	SDK	Core Flutter and testing framework
get	^4.6.6	State management and navigation (GetX)
http	^1.2.1	HTTP client for REST API calls
mockito	^5.6.4	Mock object generation for testing
build_runner	^2.15.0	Code generation for mock files

3. Test File Structure

All tests reside in the test/ directory, mirroring the lib/ structure for easy navigation:

```
test/
  models/
    weather_test.dart
    country_test.dart
    news_test.dart
  services/
    api_service_test.dart
    api_service_test.mocks.dart   ← generated by build_runner
  controllers/
    weather_controller_test.dart
    country_controller_test.dart
    news_controller_test.dart
  widgets/
    country_card_test.dart
    news_card_test.dart
  widget_test.dart               ← top-level app smoke test
  task_progress.md
```

Naming convention: all test files use the *_test.dart suffix, which Flutter's test runner discovers automatically when running flutter test.

4. Unit Tests — Models

Model tests verify that data classes correctly parse JSON responses from external APIs, handle missing or null fields gracefully, and produce accurate Dart objects.

4.1 Weather Model (weather_test.dart)

The Weather model maps OpenWeatherMap API JSON responses. Tests cover:

Test Case	Description	Status
Direct constructor	Creates Weather instance with all required fields	PASS
fromJson — valid JSON	Parses full OpenWeatherMap response correctly	PASS
fromJson — empty JSON	Throws NoSuchMethodError on empty map	PASS

fromJson — missing weather array	Throws NoSuchMethodError when weather key absent	PASS
fromJson — integer temperature	Converts integer temps to double correctly	PASS
fromJson — zero temperature	Handles 0.0° and negative feels_like values	PASS
Identical instances	Same JSON produces equal field values in two instances	PASS

Key assertion: the model uses .toDouble() conversion so both integer and decimal API responses are stored as Dart double, preventing type errors at runtime.

4.2 Country Model (country_test.dart)

The Country model maps the REST Countries API, which uses a deeply nested JSON structure:

Test Case	Description	Status
Direct constructor	Creates Country instance with all required fields	PASS
fromJson — valid JSON	Parses nested name.common, capital[0], currencies	PASS
fromJson — empty JSON	Throws NoSuchMethodError on empty map	PASS
fromJson — missing currencies	Returns empty string for currency gracefully	PASS
fromJson — null capital	Returns empty string when capital is null	PASS
fromJson — null name	Throws NoSuchMethodError when name is null	PASS
Multiple currencies	Picks first currency from currencies map	PASS

Note: The currency extraction uses (json["currencies"] as Map).values.first["name"] which returns the first currency name. Tests confirm this behavior and that null guard prevents crashes when the field is absent.

4.3 News Model (news_test.dart)

The News model maps NewsAPI article JSON, with special handling for the nested source object:

Test Case	Description	Status
Direct constructor	Creates News instance with all required fields	PASS
fromJson — valid JSON	Parses all fields including source.name	PASS
fromJson — empty JSON	Returns empty strings for all fields (null-safe)	PASS
fromJson — empty values	Handles explicitly empty strings and null source	PASS
fromJson — missing source name	Returns empty string when source.name absent	PASS
fromJson — source not a Map	Throws TypeError when source is a plain string	PASS
Identical instances	Same JSON produces equal field values in two instances	PASS

The News model uses the null-safe operator json['source']?['name'] which gracefully handles a null source object. The test confirms that an empty JSON map returns all empty strings, making the app resilient to partial API responses.

5. Unit Tests — Services

5.1 ApiService (api_service_test.dart)

The ApiService is the HTTP layer abstraction used by all domain services. It accepts an injectable http.Client, enabling full mock testing without network calls.

Mock setup using Mockito:

```
@GenerateMocks([Client])
import 'package:mockito/annotations.dart';
// Run: flutter pub run build_runner build
// Output: api_service_test.mocks.dart
```

Test Case	Description	Status
get — 200 success	Returns decoded JSON map on HTTP 200	PASS
get — 404 error	Throws Exception on non-200 status code	PASS
get — no query params	Works correctly without optional query parameters	PASS

The MockClient stub pattern used throughout all controller tests:

```
when(mockClient.get(any)).thenAnswer(
  (_) async => http.Response(jsonEncode(mockResponse), 200),
);
```

6. Unit Tests — Controllers

Controller tests verify business logic, GetX reactive state management, and error handling. Each controller test uses a shared MockClient from api_service_test.mocks.dart injected through the dependency chain: MockClient → ApiService → DomainService → Controller.

6.1 WeatherController (weather_controller_test.dart)

Test Case	Description	Status
Initial state	weather=null, isLoading=false, error=''	PASS
fetchWeather — success	Populates weather object with correct city and temp	PASS
fetchWeather — 404 failure	Sets error='City not found', weather stays null	PASS

fetchWeather — empty city	Returns immediately without HTTP call	PASS
fetchWeather — whitespace city	Trims whitespace and skips fetch	PASS
Loading state transitions	isLoading=true during fetch, false after completion	PASS
Fetch multiple cities	Weather data updates correctly on sequential calls	PASS

6.2 NewsController (news_controller_test.dart)

Test Case	Description	Status
Initial state	newsList=[], isLoading=false, error=''	PASS
fetchTopHeadlines — success	Populates newsList with correct article data	PASS
fetchTopHeadlines — 403 failure	newsList stays empty, error message set	PASS
searchNews — success	Searches and populates newsList with results	PASS
searchNews — empty/whitespace query	Falls back to fetchTopHeadlines behavior	PASS
searchNews — 404 failure	newsList empty, error message non-empty	PASS
Loading state transitions	isLoading=true during fetch, false after completion	PASS

6.3 CountryController (country_controller_test.dart)

Test Case	Description	Status
Initial state	countries=[], isLoading=false, error=''	PASS
search — success	Populates countries list with name, capital, currency	PASS
search — 404 failure	Sets error containing 'No country found'	PASS
Clear results on error	Previous successful results cleared after failure	PASS
Empty query	Controller processes empty string without crashing	PASS

Loading state transitions	isLoading=true during search, false after completion	**PASS**

7. Widget Tests

Widget tests verify UI rendering and component behavior using Flutter's WidgetTester. All widget tests wrap components in GetMaterialApp to support GetX routing and theming.

7.1 CountryCard Widget (country_card_test.dart)

The CountryCard is a reusable list item displaying country name, capital, and flag image:

Test Case	Description	Status
Name and capital render	find.text('Japan') and find.text('Tokyo') succeed	PASS
Empty flag fallback	Shows Icons.flag when flag URL is empty	PASS
Card and ListTile structure	find.byType(Card) and ListTile each find exactly one	PASS
ListTile title and subtitle	title.data == country name, subtitle.data == capital	PASS

7.2 NewsCard Widget (news_card_test.dart)

The NewsCard is a reusable list item displaying news title, source, and optional image:

Test Case	Description	Status
Title and source render	find.text(title) and find.text(source) succeed	PASS
No image when empty	find.byType(Image) finds nothing when image=''	PASS
Card and ListTile structure	Both Card and ListTile present in widget tree	PASS
Image with error builder	Image widget present for non-empty URL; title still visible	PASS

7.3 App-Level Widget Test (widget_test.dart)

Top-level smoke tests verify the entire app renders correctly:

Test Case	Description	Status

App renders without errors	MyApp() creates and pumps GetMaterialApp without crashing	PASS
Bottom navigation labels	Countries, News, Weather labels all present in nav bar	PASS
MyApp instantiation	MyApp() constructor returns non-null object	PASS

8. Mocking Strategy

8.1 Mockito Setup

The project uses Mockito 5.6.4 with Dart's null-safety support and code generation via build_runner. The annotation @GenerateMocks([Client]) on the api_service_test.dart file instructs the code generator to produce a complete MockClient class.

Build command to generate mock files:

```
flutter pub run build_runner build --delete-conflicting-outputs
```

This generates test/services/api_service_test.mocks.dart containing the MockClient class, which is then imported by all three controller test files.

8.2 Dependency Injection Pattern

The MockClient is injected through the constructor chain rather than using service locators, keeping tests hermetic:

```
setUp(() {
  mockClient = MockClient();
  final apiService = ApiService(client: mockClient);
  final weatherService = WeatherService(api: apiService);
  controller = WeatherController(service: weatherService);
});
```

This pattern has several advantages:

- No global state or GetX bindings needed in tests
- Each test gets a fresh MockClient preventing test pollution
- The full service chain is exercised, not just isolated units
- HTTP responses are fully controlled without any network calls

8.3 Mock Response Patterns

Controller tests use two primary mock patterns:

Success pattern:

```
when(mockClient.get(any)).thenAnswer(
  (_) async => http.Response(jsonEncode(mockResponse), 200),
);
```

Failure pattern:

```
when(mockClient.get(any)).thenAnswer(
  (_) async => http.Response('Not Found', 404),
);
```

9. Test Coverage Analysis

9.1 Coverage by Layer

Layer	Files Tested	Test Count	Coverage Notes
Models	3 / 3	~20 tests	Full JSON parsing, null handling, edge cases
Services	1 / 4	3 tests	ApiService tested; domain services covered via controller tests
Controllers	3 / 3	~19 tests	Success, failure, loading states, input validation
Widgets	2 / 2 cards + App	11 tests	Rendering, structure, icon fallbacks, image handling
Screens	0 / 5	0 tests	Identified as future improvement area

9.2 Generating Coverage Report

Run the following commands to generate and view a coverage report:

```
# Run all tests with coverage
flutter test --coverage

# Generate HTML report (requires lcov)
genhtml coverage/lcov.info -o coverage/html

# Open in browser (macOS)
open coverage/html/index.html
```

The lcov.info file is generated at coverage/lcov.info and can be integrated with CI tools or IDE coverage viewers (VS Code Coverage Gutters, IntelliJ coverage runner).

9.3 Coverage Gaps and Recommendations

The following areas are not yet covered and represent opportunities for improvement:

- Screen tests: WeatherScreen, CountriesScreen, NewsScreen, HomeScreen require GetX controller initialization in tests — use Get.put() inside pumpWidget

- Integration tests: End-to-end flows using flutter_test's integrationDriver or patrol package
- CountryService, NewsService, WeatherService: Direct unit tests for each service's API URL construction and response mapping
- Navigation tests: Verify that tapping CountryCard navigates to CountryDetailsScreen using GetX routing mocks
- Error UI tests: Confirm that error states display appropriate messages in screen widgets

10. How to Run Tests

10.1 Prerequisites

- Flutter SDK installed (Dart ^3.11.5)
- Run flutter pub get to install all dependencies
- Generate mocks: flutter pub run build_runner build --delete-conflicting-outputs

10.2 Run Commands

Command	What it does
flutter test	Run all tests in the test/ directory
flutter test --coverage	Run tests and generate coverage/lcov.info
flutter test test/models/	Run only model tests
flutter test test/controllers/	Run only controller tests
flutter test test/widgets/	Run only widget tests
flutter test -v	Verbose output showing each test case name

10.3 IDE Integration

Both VS Code and Android Studio / IntelliJ support running individual tests with a click:

- VS Code: Click the green play button next to test() or group() in any test file
- VS Code: Install Flutter extension and Coverage Gutters for inline coverage display
- Android Studio: Right-click a test file and select Run tests in <filename>
- Android Studio: Use Run > Edit Configurations to set up coverage runs

11. Task Progress

Based on the task_progress.md file found in the project:

Task	Status	Notes
Analyze codebase structure	DONE	All models, controllers, services reviewed

Add mockito and build_runner dev dependencies	DONE	Already in pubspec.yaml
Create unit tests for models (Country, News, Weather)	DONE	20+ test cases
Create unit tests for services	PARTIAL	ApiService done; domain services pending
Create unit tests for controllers with mocks	DONE	All 3 controllers tested
Create widget tests for CountryCard and NewsCard	DONE	4 tests each
Create widget tests for main screens	PENDING	Planned next step
Update existing widget_test.dart	DONE	3 smoke tests added
Create utility helper for testable functions	PENDING	Planned
Run all tests and verify they pass	PENDING	Requires build_runner run first
Generate test coverage report	PENDING	Use flutter test --coverage
Create README with testing guide	PENDING	Planned final deliverable

12. Testing Best Practices Applied

12.1 Test Organization

- group() blocks used to logically organize related tests by model, method, or behavior
- setUp() used in all controller tests to create fresh instances before each test
- Naming follows the should <action> when <condition> convention for readability
- Test files mirror the lib/ directory structure for easy discoverability

12.2 Assertions

- expect(value, matcher) used throughout rather than assertions that throw
- isNull / isNotNull matchers used for optional fields
- isEmpty / isNotEmpty used for collections over length == 0
- throwsA(isA<NoSuchMethodError>()) used to document and verify expected failure modes
- isA<TypeError>() used where type casting errors are expected

12.3 Widget Test Setup

- GetMaterialApp used instead of MaterialApp to support GetX features in tests
- Scaffold wrapper prevents Material widget ancestor errors
- find.byType() used for structural assertions, find.text() for content assertions
- tester.widget<T>() used to extract widget instances for property inspection

12.4 Mock Design

- MockClient injected via constructor, not overriding global state
- when(...).thenAnswer((_) async => ...) used for async HTTP stubs
- any matcher used for URL matching to keep tests focused on response handling
- Sequential when() calls override previous stubs within the same test

13. Learning Resources

The following YouTube resources are recommended for deepening Flutter testing knowledge:

Search Term	Channel	Focus Area
Flutter testing tutorial	Reso Coder	Comprehensive overview
Unit testing Flutter	Flutter Official	Unit test fundamentals
Widget testing Flutter	Reso Coder	UI component testing

Flutter test coverage	Vandad Nahavandipoor	Coverage tools
Testing best practices Flutter	Flutter Official	Patterns and pitfalls

Appendix: Test Summary Table

Test File	Group	# Tests	Type
weather_test.dart	Weather Model	7	Unit — Model
country_test.dart	Country Model	7	Unit — Model
news_test.dart	News Model	7	Unit — Model
api_service_test.dart	ApiService	3	Unit — Service
weather_controller_test.dart	WeatherController	7	Unit — Controller
news_controller_test.dart	NewsController	7	Unit — Controller
country_controller_test.dart	CountryController	6	Unit — Controller
country_card_test.dart	CountryCard Widget	4	Widget Test
news_card_test.dart	NewsCard Widget	4	Widget Test
widget_test.dart	MyApp / App Init	3	App Smoke Test
TOTAL		**55**	**8 files**

End of Report